# Software Engineering

## Lecture 3: Requirements Engineering

**Prof. Nien-Lin Hsueh**
Department of Information Engineering and Computer Science
Feng Chia University

# Chapter 3: Roadmap & Core Curriculum

## Part 1: Foundations, Classification & Metrics

- **3.1 Foundations of Requirements Engineering:** What is RE, cost curve of late defect fixing, user vs. system requirements.
- **3.2 Stakeholders & Requirements Classification:** Stakeholder personas, functional requirements, and domain business rules.
- **3.3 Non-Functional Requirements & Metrics:** Product, organizational, external qualities (FURPS+), and verifiable metrics.

## Part 2: Elicitation, Validation & AI Era

- **3.4 Requirements Elicitation & Use Cases:** 4-stage elicitation loop, scenarios, UML use case diagrams, and Jacobson specifications.
- **3.5 Requirements Validation & Management:** 5 validation pillars, requirements reviews, traceability matrix, and RFC change control.
- **3.6 AI-Assisted Requirements Engineering:** Generative AI copilots, user story refinement, ambiguity detection, human-in-the-loop.
- **3.7 Synthesis & Conceptual Recap:** Key takeaway review, fill-in-the-blank quiz, discussion questions, and references.

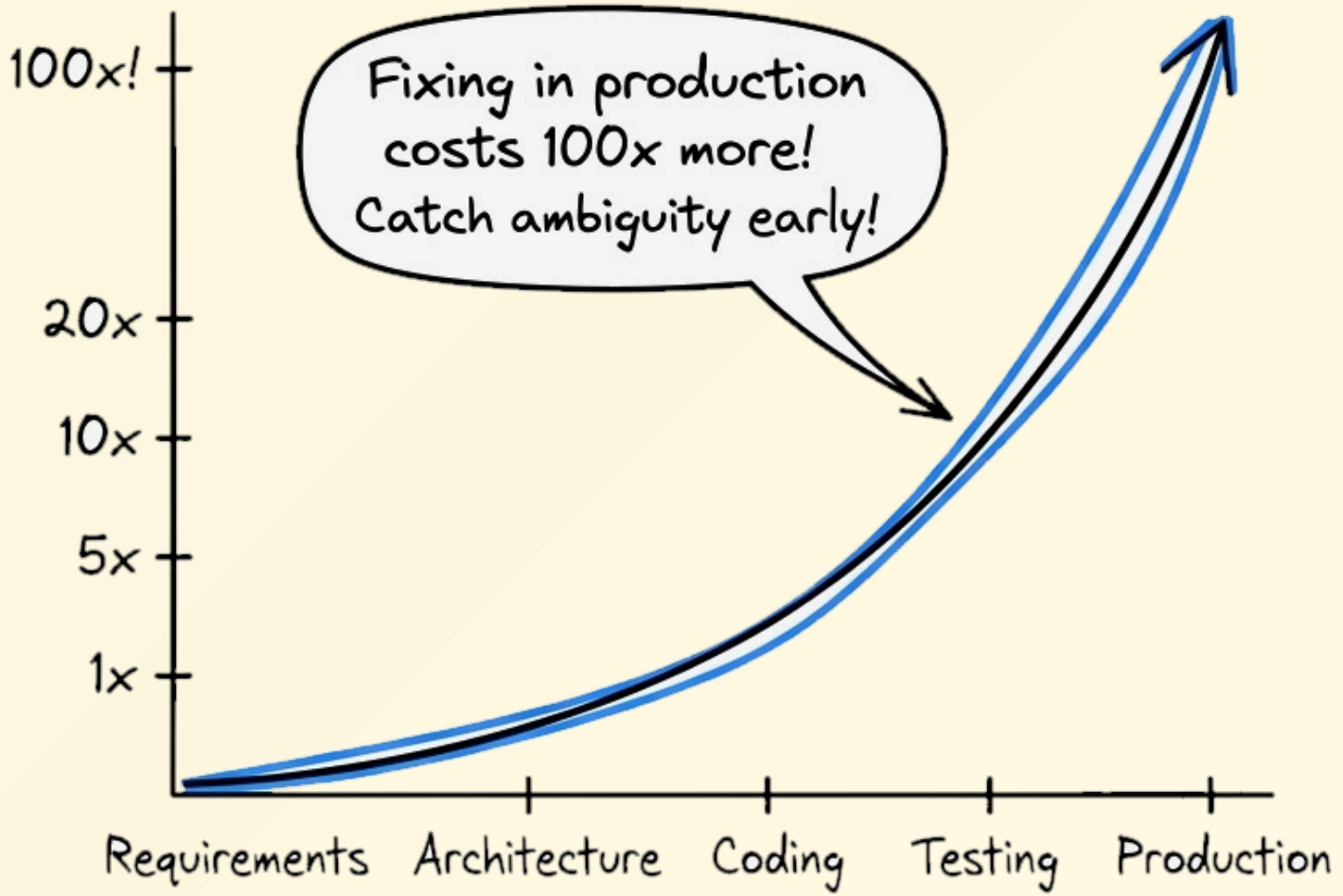
# Focus Questions

- What is **Requirements Engineering (RE)** and why is fixing requirements errors early so critical?
- What is the fundamental difference between **User Requirements** and **System Requirements**?
- How do **Functional**, **Non-Functional**, and **Domain Requirements** differ?
- What are the 4 main stages of the **Requirements Elicitation & Analysis** process?
- How can non-functional goals be transformed into **verifiable, testable metrics**?
- How do modern **Large Language Models (LLMs)** enhance the RE process, and what are the human-in-the-loop safeguards?

# 3.1 Foundations of Requirements Engineering

> *"The hardest single part of building a software system is deciding precisely what to build."*
> *— Fred Brooks, No Silver Bullet (1987)*

# Exponential Cost of Requirements Errors



# Foundations of Requirements Engineering

> *The **systematic** process of establishing the services that a customer **requires** from a system and the operational/developmental **constraints** under which it operates.*

- **Dual Function of Requirements:**
  1. **Contract Bidding:** High-level abstract statements open to contractor interpretation.
  2. **Contract Basis:** Detailed functional specifications defining precisely what must be delivered.
- **The Fundamental Goal:**
  - Bridging the gap between the messy, ambiguous real world of human stakeholders and the precise, deterministic world of software systems.

## User Requirement (Customer Intent)

The MHC-PMS shall generate monthly summary reports of drug costs prescribed by clinics

## Engineering Translation



## System Requirements (Developer Contract)

- 1.1: Triggered on last working day of month
- 1.2: Standard layout: drug name, doses, total cost
- 1.3: Handles 10k records < 60s without DB lock

# Types of Requirements

- **User Requirements:**
  - High-level abstract statements written in natural language with diagrams.
  - Target audience: **Clients, system users, managers, non-technical stakeholders.**
  - Focuses on **what** services the system provides from the user perspective.
- **System Requirements:**
  - Detailed, structured specifications defining system functions, services, and operational constraints precisely.
  - Target audience: **Developers, software architects, system testers.**
  - Serves as the baseline technical contract for system *implementation*.
- **Domain Requirements:**
  - Constraints and operational rules derived directly from the application domain (e.g., industry regulations, physical laws).
  - Non-negotiable boundaries that often override both user preferences and technical designs.

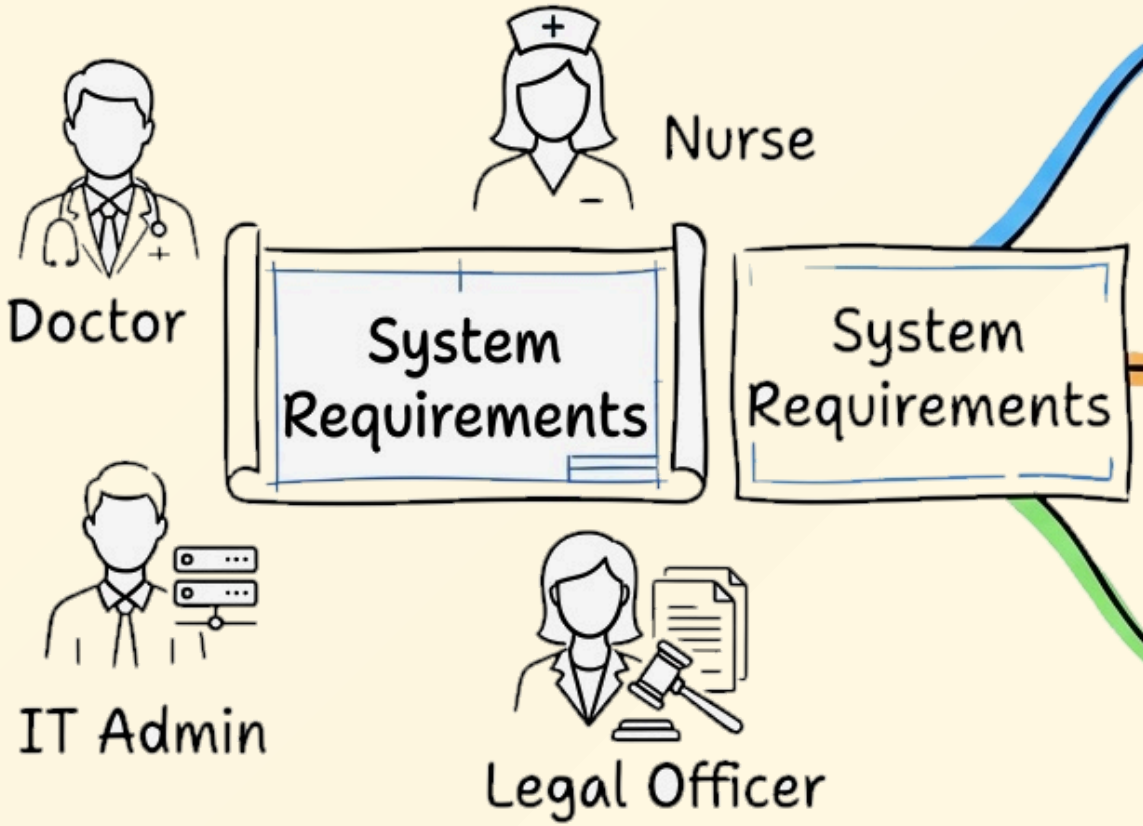
# A Healthcare Example

- **User Requirement (High-Level Intent):**

> *"The MHC-PMS shall generate monthly summary reports showing the cost of drugs prescribed by each clinic."*

- **System Requirements (Detailed Technical Specification):**
  - **1.1:** On the last working day of each month, a summary report shall be generated of drug costs across each registered clinic.
  - **1.2:** The system shall produce reports using a standardized layout showing drug names, total doses prescribed, and total cost.
  - **1.3:** If drug cost data exceeds 10,000 records, generation shall complete within 60 seconds without locking the transactional database.
- **Domain Requirements (Healthcare Mandates):**
  - **D.1 (Patient Privacy):** Per health data laws (e.g., HIPAA), cost reports must aggregate data so individual patient identities cannot be deduced.
  - **D.2 (Formulary Standards):** Drug classifications and dosage units must strictly follow national medical formulary standards (e.g., FDA NDC / BNF).

# The Three Types of Software Requirements




**Functional Requirements:**
Specific services, behaviors, calculations, user actions

**Non-Functional Requirements:**
Quality attributes, response time, security, 99.9% uptime

**Domain Requirements:**
Industry laws, healthcare compliance, physical constraints

# Concept Check Question 1

In requirements engineering, what is the critical operational distinction between **User Requirements** and **System Requirements**?

- **A.** User requirements are high-level stakeholder goals; system requirements are detailed functional contracts for developers.
- **B.** User requirements specify UI wireframes; system requirements specify backend database schemas.
- **C.** User requirements can never change; system requirements are refactored continuously during daily scrums.
- **D.** User requirements come from external legal auditors; system requirements are generated by compiler tools.



# 3.2 System Stakeholders & Requirements Classification

> *"Software systems have many masters. If you fail to identify who has a stake, their unspoken expectations will become your project's failure."*

# System Stakeholders & Requirements Classification

> *Any person, system or organization affected by the system who has a legitimate interest in its behavior and operational outcome.*
>
> ”

- **Types of Stakeholders:**
  - **End Users:** Patients, Doctors, Nurses, Receptionists.
  - **System Managers:** Clinic Managers, IT Directors, Compliance Officers.
  - **External Stakeholders:** Health Authorities, Medical Device Regulators, Insurance Payors.
- **The Stakeholder Conflict Challenge:**
  - Different stakeholders have *competing*, *conflicting* requirements (e.g., Doctors want rapid data entry; Compliance Officers want comprehensive multi-factor audit logging).

# Case Study: Stakeholder Ecosystem in UberEats

- **The Multi-Sided Stakeholder Ecosystem:**
  - **Hungry Customer (Eater):** Demands fast delivery, accurate ETA, live GPS tracking, and friction-free refunds.
  - **Restaurant Merchant:** Wants predictable kitchen prep buffers, timely courier arrivals, and locked payments once cooking starts.
  - **Delivery Courier:** Demands fair dispatching algorithms, transparent tip payouts, short distances, and safe parking tolerance.
  - **Platform Operations (Uber):** Optimizes network liquidity, balances supply/demand via dynamic fees, and prevents promotional fraud.
  - **External Regulators:** Enforce food safety temperature standards, labor regulations, and mandatory electronic fiscal receipts.
- **The Stakeholder Conflict Challenge:**
  - *Customer vs. Restaurant:* Customer cancels 5 minutes after ordering; restaurant already started cooking expensive ingredients. Who absorbs the loss?

# UberEats: The Requirements Triad in Practice

> *The Requirements Triad balances user-facing services, operational qualities, and environmental compliance.*

## 1. Functional

### Services & Behavior

- Filter restaurants by dietary restrictions and live delivery ETA.
- Broadcast delivery jobs to optimal couriers based on GPS location.
- Automatically split and transfer payouts upon order delivery.

## 2. Non-Functional

### Quality & Constraints

- **Performance:** Courier GPS coordinates refresh on map $\leq 2$ seconds.
- **Availability:** Maintain $\geq 99.99\%$ uptime during peak dinner hours.
- **Security:** Card credentials strictly comply with PCI-DSS Level 1.

## 3. Domain

### Environmental Mandates

- **Food Safety:** Perishable delivery duration complies with health agency rules.
- **Fiscal Law:** Generate legally compliant electronic VAT invoices.
- **Labor Law:** Comply with local limits on courier shift intervals.

# Functional Requirements: Challenges

- **Requirements Imprecision:**
  - Ambiguous natural language requirements lead to serious engineering misunderstandings.
  - *UberEats Example:* "Customer may cancel an order before delivery."
    - *Customer Interpretation:* Cancel anytime with full refund while food is in transit.
    - *Restaurant Interpretation:* No cancellation once the kitchen starts cooking.
- **Completeness vs. Consistency:**
  - **Completeness:** All edge cases and services must be defined (e.g., courier vehicle breakdown mid-delivery, restaurant out-of-stock substitutions).
  - **Consistency:** Requirements must not contradict each other (e.g., "Full refund upon cancellation" vs. "Guaranteed merchant compensation once food is prepared").
  - *Reality Check:* In complex platforms like UberEats, achieving 100% upfront completeness is practically impossible, requiring continuous iterative refinement.

# Concept Check Question 2

Consider the following specification for the UberEats platform:

> *"Due to municipal food hygiene regulations, perishable warm food delivery transit time shall not exceed 45 minutes, and containers must maintain a temperature above 60°C throughout transit."*

What category of software requirement does this statement represent?

- **A.** Domain Requirement (a constraint imposed by the operational environment, industry regulations, or physical laws)
- **B.** Functional Requirement (a specification of an active software computation, user feature, or system service)
- **C.** Non-Functional Requirement (a general software quality attribute concerning performance, scalability, or uptime)
- **D.** User Requirement (a high-level, natural-language goal or business vision expressed by end consumers)



# Interactive Activity: Cybercab Requirements Triad (Pair Discussion)

> ***Scenario:*** *Suppose you are Elon Musk. Your autonomous* ***Cybercab*** *is officially street-legal. Beyond personal commuting, you can dispatch it to an autonomous robotaxi fleet to earn rental income.*

## 1. Functional

*User features & vehicle services:*

- **Guiding Questions:**
  - How does a rider summon, track, and enter the Cybercab?
  - How does the owner toggle between personal drive and fleet rental?
  - How does the system calculate fares and disburse rental revenue?

## 2. Non-Functional

*Operational qualities & safety limits:*

- **Guiding Questions:**
  - What reaction latency limit is acceptable for obstacle braking?
  - What uptime availability must the cloud dispatch fleet maintain?
  - What cybersecurity safeguards prevent remote fleet takeovers?

## 3. Domain

*Mandatory legal & industry rules:*

- **Guiding Questions:**
  - What federal autonomous vehicle certifications (e.g. NHTSA) apply?
  - What crash investigation data (blackbox/EDR) is legally required?
  - What commercial passenger liability insurance is mandated by law?

# 3.3 Non-Functional Requirements (NFR) & Metrics

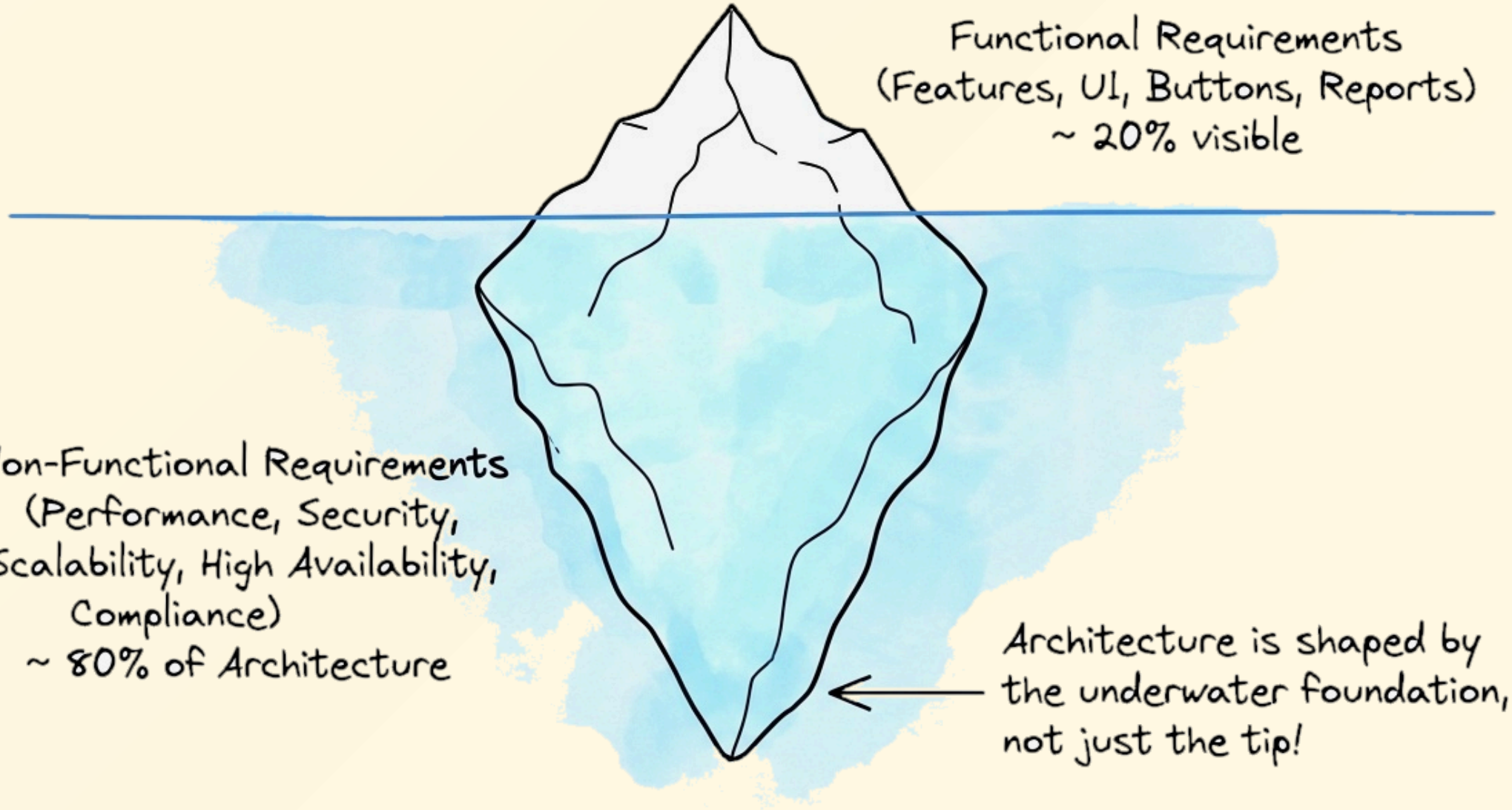
> *"If you can't measure it, you can't manage it — and you certainly cannot verify it."*
> *— Tom DeMarco*

# Non-Functional Requirements & Metrics

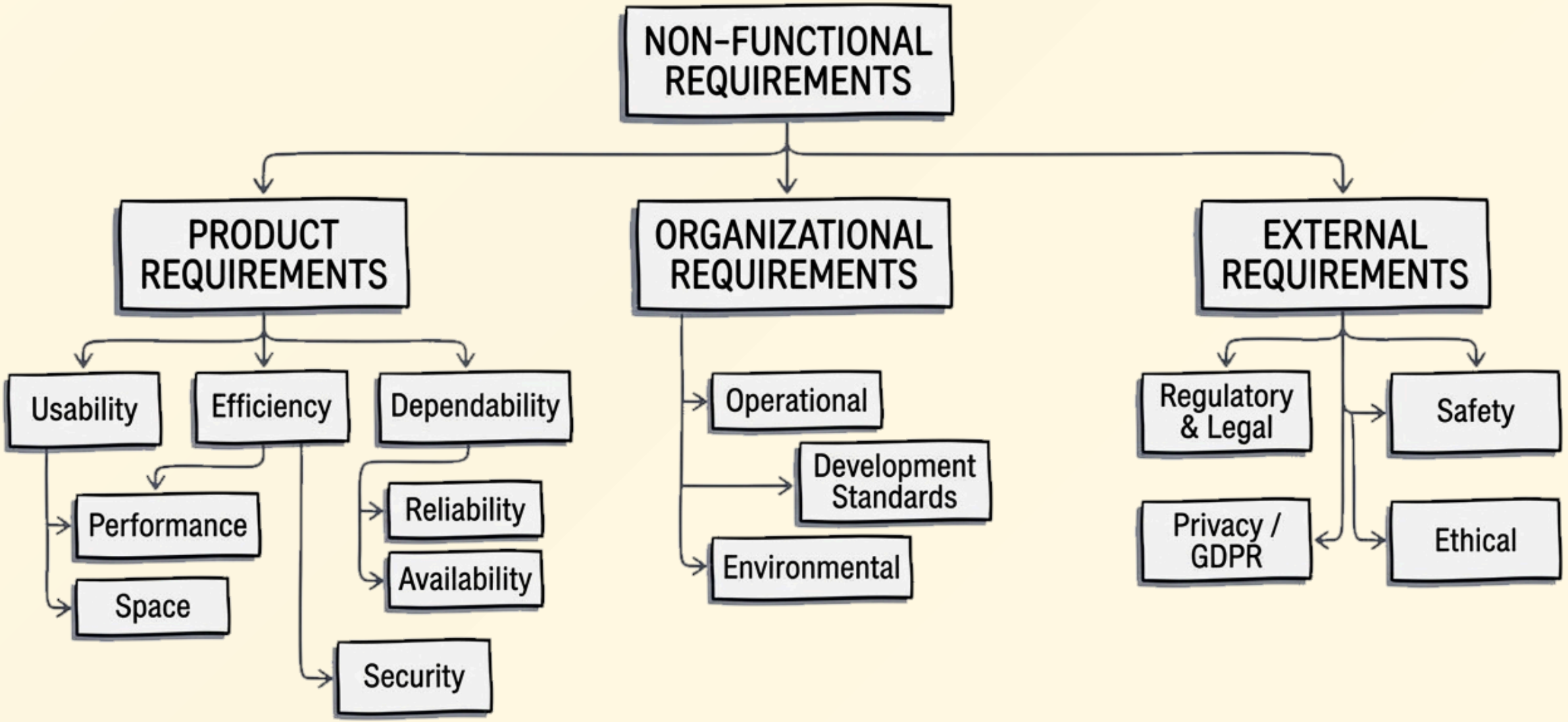
> *Constraints on the services or functions offered by the system, including timing constraints, security, reliability, and standards compliance, fundamentally shaping software architecture.*
>
> ”

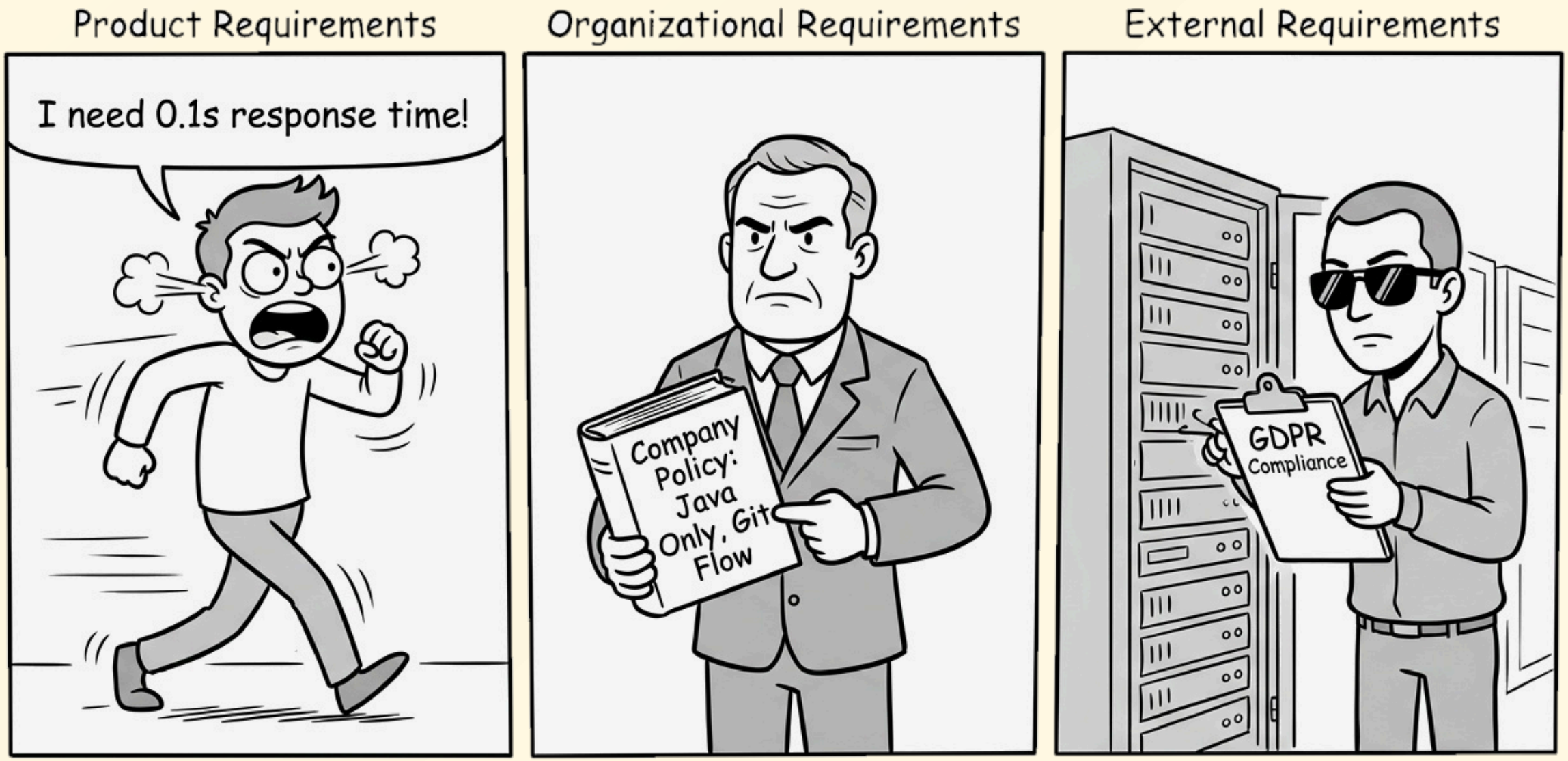
- **System-Wide Impact:**
  - NFRs often apply to the system as a **whole** rather than **individual** features.
  - Failing to meet an NFR can render the entire system useless (e.g., a slow or insecure medical database).
- **Architectural Influence:**
  - Performance NFRs dictate module communication patterns; security NFRs introduce authentication infrastructure.
  - *Architecture is shaped far more by NFRs than by functional features!*

# Non-Functional Requirements Iceberg



# NON-FUNCTIONAL REQUIREMENTS (NFR) HIERARCHY




Product Requirements
I need 0.1s response time!
Organizational Requirements
Company Policy: Java Only, Git Flow
External Requirements
GDPR Compliance

# Classification of NFRs: Three Major Families

> *Non-functional requirements derive from product runtime behavior, internal organizational policies, and external operating environments.*

## 1. Product Requirements

**Definition:** Specify how the delivered software product must execute and behave at runtime.

- **Scope:** Runtime performance, dependability, and user experience constraints.
- **Sub-types:** Usability, Efficiency (speed/memory), Reliability, Security.
- **Examples:** Response time $\leq 2$s, availability $\geq 99.99\%$, AES-256 encryption.

## 2. Organizational Requirements

**Definition:** Consequences of policies, procedures, and standards within the customer or developer organization.

- **Scope:** Development lifecycle, operational governance, and infrastructure choices.
- **Sub-types:** Development standards, Operational runbooks, Platform constraints.
- **Examples:** Mandated Git Flow, daily offsite DB backup, target AWS deployment.

## 3. External Requirements

**Definition:** Requirements derived from factors external to the system and its engineering process.

- **Scope:** Regulatory compliance, legal obligations, safety laws, and societal ethics.
- **Sub-types:** Legislative mandates, Safety regulations, Ethical standards.
- **Examples:** GDPR privacy compliance, electronic tax invoicing, medical fail-safe rules.

# Goals vs. Verifiable NFRs

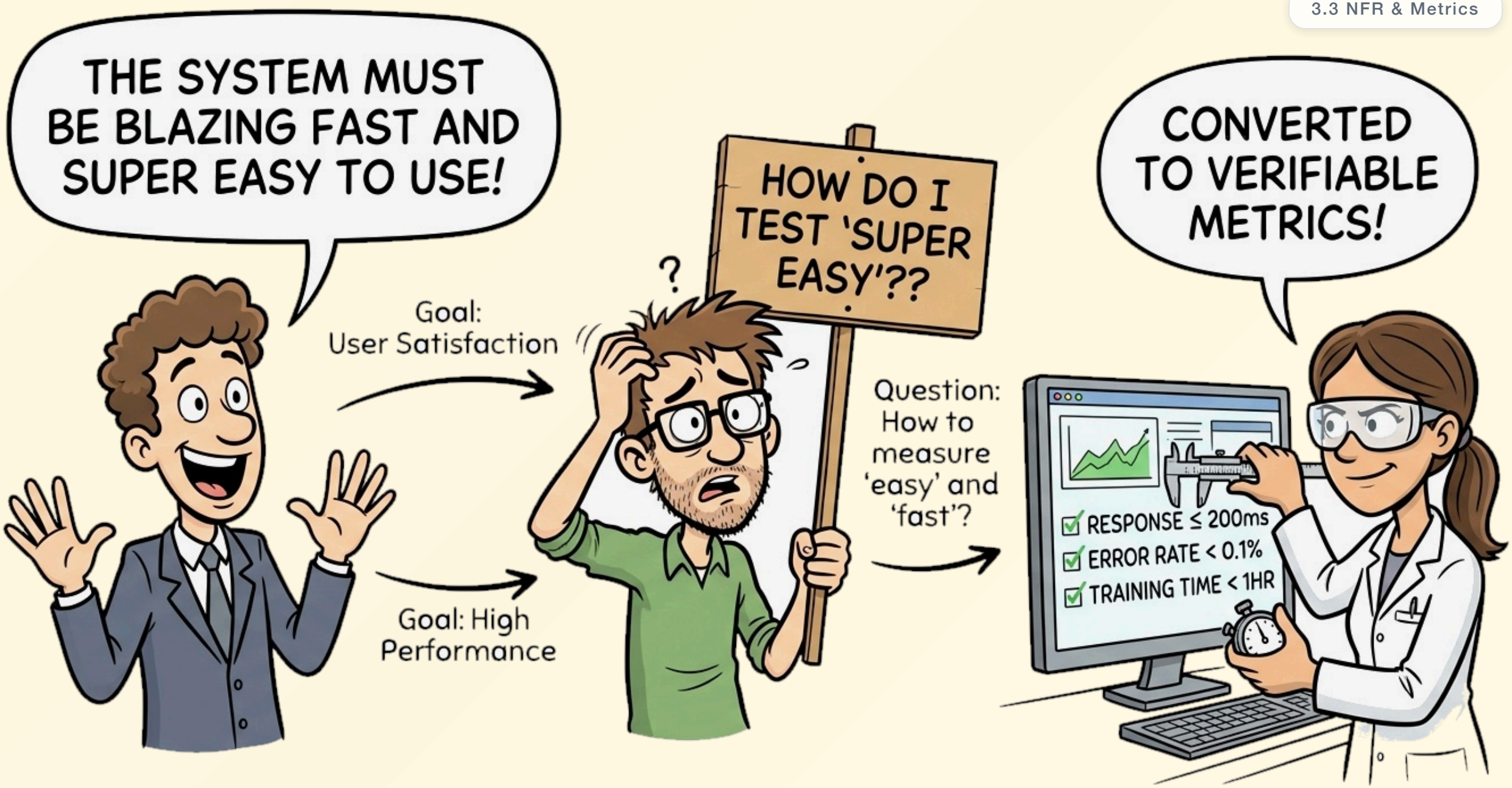
- **Goal (Imprecise Intention):**

> *"The system should be easy to use by medical staff and minimize user errors."*

- **Verifiable NFR (Testable Metric):**

> *"Medical staff shall be able to use all core functions after 2 hours of training. After training, the average number of operator errors shall not exceed 2 per working day."*

- **The Golden Engineering Principle:**
  - Always transform subjective stakeholder goals into **quantifiable, objectively testable metrics!**


THE SYSTEM MUST BE BLAZING FAST AND SUPER EASY TO USE!
Goal: User Satisfaction
Goal: High Performance
?
HOW DO I TEST 'SUPER EASY'??
Question: How to measure 'easy' and 'fast'?
CONVERTED TO VERIFIABLE METRICS!
RESPONSE ≤ 200ms
ERROR RATE < 0.1%
TRAINING TIME < 1HR

# Metrics for Specifying NFRs

| Property | Measurable Metric Examples |
|---|---|
| **Speed** | Processed transactions/sec; Response time; Screen refresh rate |
| **Size** | Megabytes of RAM; Number of ROM chips required |
| **Ease of Use** | Training time to competency; Number of help desk queries/week |
| **Reliability** | Mean Time Between Failures (MTBF); Rate of failure occurrence |
| **Robustness** | Time to restart after failure; Percentage of data corrupted on power loss |
| **Portability** | Percentage of target-dependent statements; Number of supported OS platforms |

# Concept Check Question 3

A client provides an imprecise non-functional goal: "*The order checkout system must be blazing fast and highly reliable.*" Which of the following correctly transforms this vague goal into a **verifiable, testable engineering metric**?

- **A.** 99% of checkouts shall have response time $\leq 500$ ms, and peak uptime shall be $\geq 99.95\%$.
- **B.** The checkout UI shall use sleek animations so customers perceive maximum speed.
- **C.** All checkout services shall use memory-safe code to guarantee bug-free execution.
- **D.** The cloud database shall allocate unlimited RAM whenever transaction load increases.



# Interactive Activity: Quantifying Cybercab NFRs (Pair Discussion)

> ***Scenario:*** *Elon Musk demands: "The Cybercab must offer the safest drive, the smoothest rental experience, and the most effortless automated return." How do you translate these desires into* ***verifiable, testable NFR metrics****?*

## 1. Drive Safety & Latency

*Turning "safe drive" into metrics:*

- **Guiding Questions:**
  - What is the max perception-to-braking latency in milliseconds ($\text{ms}$)?
  - Under what adverse weather conditions (fog, night, rain) must this hold?
  - What statistical percentile (e.g. 99.99%) guarantees passenger safety?

## 2. Renting & Pickup Flow

*Turning "smooth rental" into metrics:*

- **Guiding Questions:**
  - How fast must cloud dispatch match and route a nearby vehicle ($\leq N$ s)?
  - What is the max door unlock latency via Bluetooth/NFC upon renter arrival?
  - What is the allowable False Rejection Rate (FRR) for passenger identity check?

## 3. Return & Inspection

*Turning "effortless return" into metrics:*

- **Guiding Questions:**
  - How fast must cabin cameras/sensors detect left-behind items or trash?
  - What minimum battery reserve must remain before permitting rental completion?
  - Within how many seconds must damage deposit hold and billing finalize?

# Interactive Activity: AI Requirements Prompting (Pair Discussion)

> ***Scenario:*** *Work in pairs with the same raw concept: "A Food Delivery app connecting Customers, Restaurants, and Couriers for fast doorstep delivery." Compare how prompt structure radically alters AI specification quality:*

## Student A: Direct Naive Request

*Unstructured prompt without guidance:*

- **Your Prompt:** "*Generate the requirements specification for this food delivery system.*"
- **Inspect What AI Produces:**
  - Generic feature bullets with loose boundaries.
  - Vague, untestable NFRs ("*blazing fast*", "*highly secure*", "*super intuitive*").
  - Merges customer user needs with backend DB details.
  - Ignores food hygiene, tax, and domain laws.

## Student B: Structured RE Request

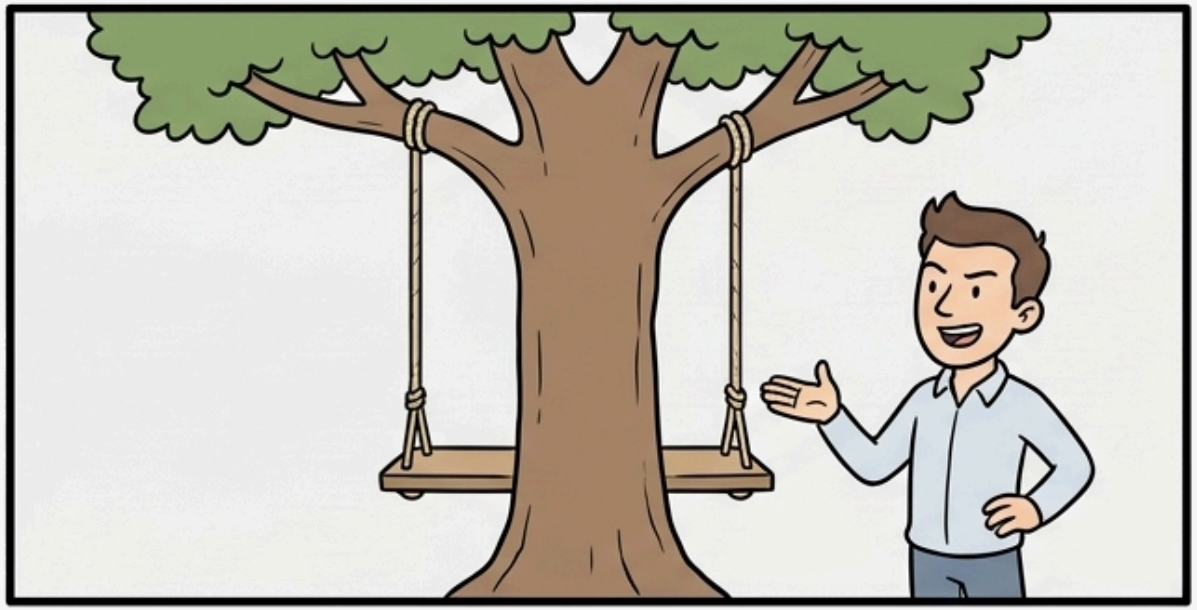
*Multi-dimensional RE framework prompt:*

- **Your Prompt Demands:**
  - **FR vs. NFR:** Decompose NFRs into verifiable metrics ( $\leq 500$ ms latency, $99.95\%$ uptime).
  - **User vs. System:** Split end-user goals from technical platform contracts.
  - **Domain Rules:** Enforce PCI-DSS payment and food hygiene regulations.
- **Compare:** How much more rigorous is Output B?

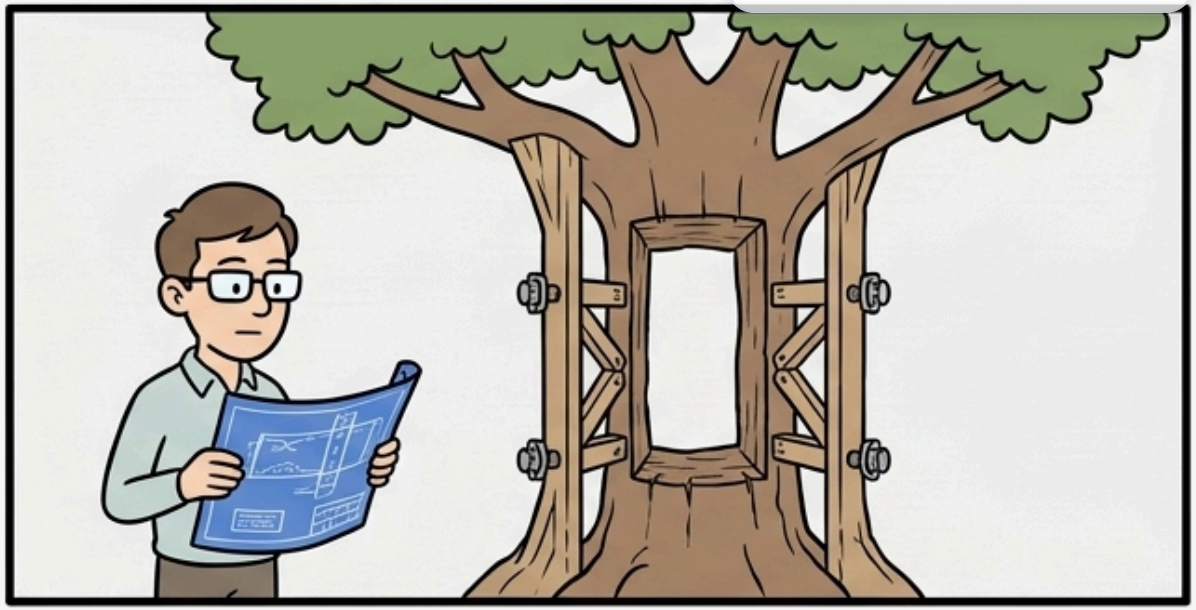
# 3.4 Requirements Elicitation & Modeling

> *"Customers do not know what they want until you show it to them — elicitation is a collaborative journey of discovery."*

What the customer described



How the systems analyst designed it



How the programmer coded it



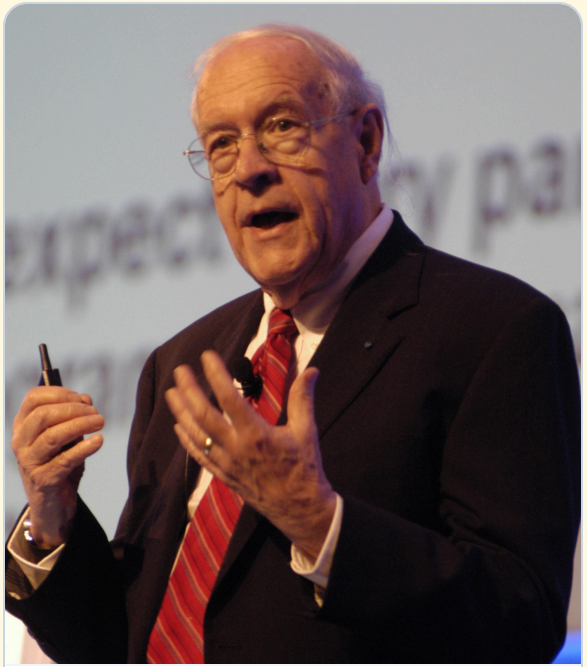
What the customer actually wanted



# Why Requirements Elicitation is Hard

> *"The hardest single part of building a software system is deciding precisely what to build."*
> — ***Fred Brooks***, *No Silver Bullet (1987)*

- **1. Tacit Knowledge Barrier:**
  - Experts struggle to articulate routine habits ("*It's just obvious!*").
- **2. Solution Bias (Premature Anchoring):**
  - Non-technical clients jump straight to UI/tech solutions.
- **3. Conflicting Stakeholder Agendas:**
  - Competing priorities across marketing, security, and finance.
- **4. Requirements Volatility:**
  - Needs continuously shift as soon as prototypes are touched.

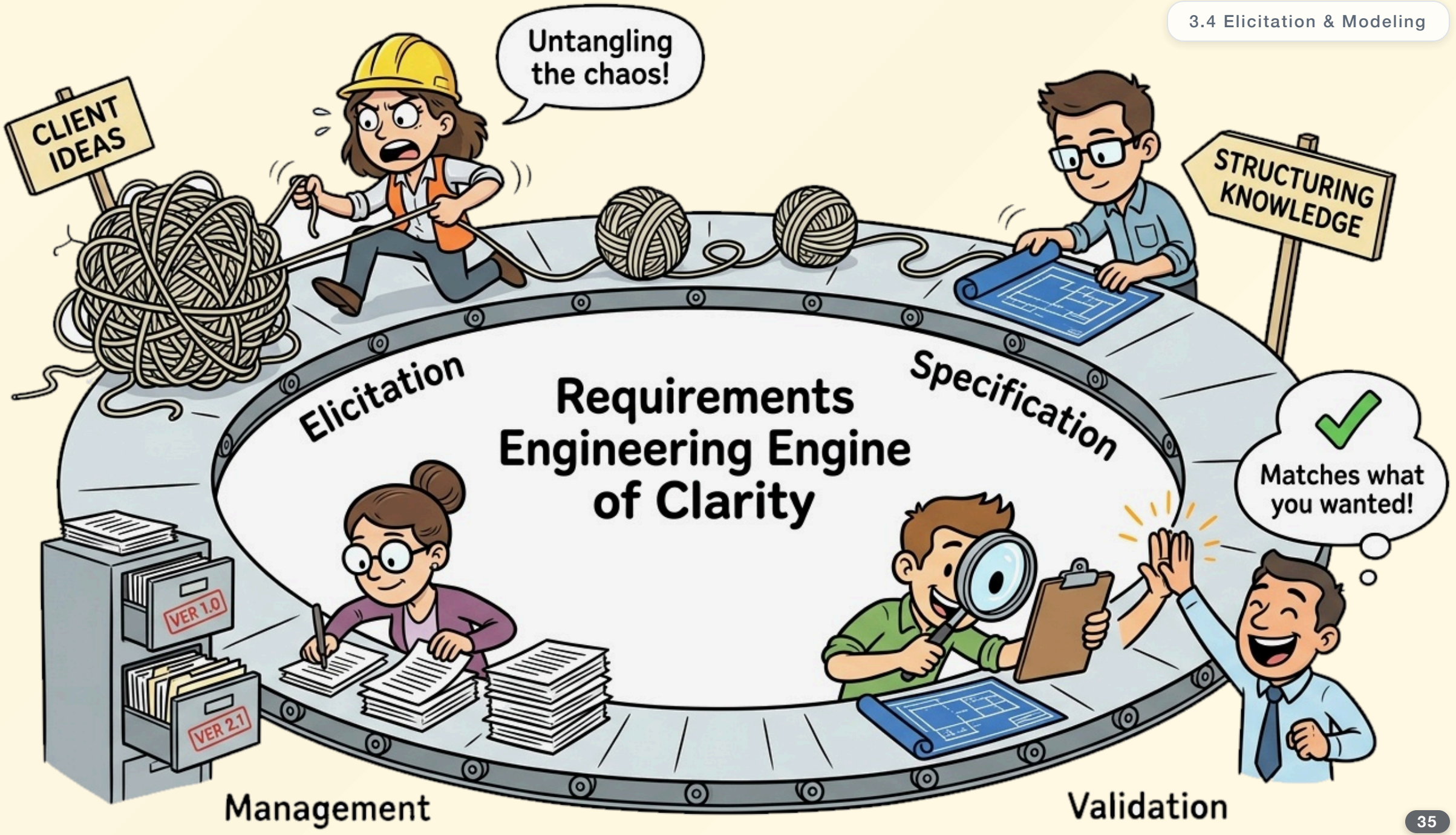


**Fred Brooks (1931–2022)**
Turing Award (1999) / Wikipedia

# Requirements Elicitation & Modeling

> *The collaborative, iterative process of discovering, understanding, negotiating, and modeling stakeholder needs and operational system boundaries.*

- **Four Interleaved Activities:**
  1. **Elicitation & Analysis:** Discovering requirements from stakeholders.
  2. **Specification:** Documenting requirements formally.
  3. **Validation:** Checking that requirements reflect real needs.
  4. **Management:** Managing changes to requirements over time.
- **Spiral & Iterative Nature:**
  - RE is **never a single-pass, linear activity**; it is an iterative spiral where understanding deepens across successive releases.


Untangling the chaos!
CLIENT IDEAS
STRUCTURING KNOWLEDGE
Elicitation
Requirements Engineering Engine of Clarity
Specification
Matches what you wanted!
VER 1.0
VER 2.1
Management
Validation

# Elicitation Technique 1: Stakeholder Interviews & Workshops

- **Types of Interviews:**
  - **Closed Interviews:** Pre-defined question sets useful for verifying factual constraints and compliance checklists.
  - **Open Interviews:** Unstructured, exploratory conversations uncovering stakeholder frustrations, mental models, and broad visions.
  - **Semi-Structured Interviews:** Core agenda with flexibility to drill down into unexpected operational friction points.
- **The "Unstated Assumptions" Trap:**
  - Stakeholders frequently omit obvious domain knowledge ("*Of course everyone knows taxes change by county!*").
- **Facilitated Workshops:**
  - Gathers conflicting stakeholders (e.g., Doctors, Nurses, IT, Billing) in one room with a neutral facilitator to resolve competing needs in hours rather than months.

# Interview Heuristics: Principles, Myths

- **Core Principles:**
  - **Listen 80%, Talk 20%:** Let stakeholders articulate operational pain points without interruption.
  - **Focus on Problems, Not Solutions:** Ask "*What goal are you trying to accomplish?*" rather than "*Do you want a dropdown button?*"
- **Common Myths:**
  - *Myth 1:* "Users know what they want and state it clearly." (Reality: They confuse symptoms with root causes).
  - *Myth 2:* "One interview yields a complete spec." (Reality: Interviews provide initial hypotheses).
  - *Myth 3:* "Discuss technical architecture with users." (Reality: Microservices/SQL talk alienates stakeholders).

# Elicitation Technique 2: Ethnography & Workplace Observation

> *"What people say they do, and what they actually do, are often entirely different."*

- **The Power of Direct Observation:**
  - Spending time in the actual operational environment (e.g., hospital ER, trading floor, warehouse loading dock).
- **Uncovering "Tacit Knowledge":**
  - **Tacit Knowledge:** Deeply ingrained habits, shortcuts, and informal workarounds that users perform automatically but never mention in meetings.
- **Observation Styles:**
  - **Passive Observation ("Fly on the Wall"):** Watching natural workflows without interruption to identify operational bottlenecks and cognitive load.
  - **Contextual Inquiry (Active Shadowing):** Observing users perform real tasks, asking clarifying questions in the moment ("*Why did you write that code on a sticky note?*").

# Elicitation Technique 3: Scenarios, User Stories & Storyboards

- **Concrete Narrative Scenarios:**
  - Rich, chronological walkthroughs of realistic, day-in-the-life system usage under both typical conditions and crisis situations.
- **Agile User Stories & Acceptance Criteria:**
  - **Format:** "*As a [User Role], I want [Capability], so that [Business Value].*"
  - **Given-When-Then (Gherkin):**
    - *Given* a customer with an active promo code,
    - *When* they apply the code to an eligible restaurant order,
    - *Then* the subtotal must reflect a 20% discount up to a maximum of $10.
- **Low-Fidelity Storyboarding & Paper Prototyping:**
  - Quick comic-strip sketches of user journeys that elicit early, visceral feedback before writing code ("*I didn't realize I'd need 5 screen taps just to reorder!*").


Agile User Story
As a Hungry Customer, I want live GPS tracking of my UberEats courier, so that I don't wait in the cold rain!


Uber Eats


9:41
GIVEN courier is within 2 minutes,
WHEN app alerts customer,
THEN food is delivered hot and fresh!


Acceptance Criteria Met
Uber Eats

# Elicitation Technique 4: Document Analysis & System Archaeology

- **Mining Existing Operational Artifacts:**
  - **Standard Operating Procedures (SOPs):** Official enterprise rules, compliance manuals, and safety checklists.
  - **Legacy Code & Database Schemas:** Reverse-engineering existing production systems to extract implicit business algorithms.
  - **Forms & Spreadsheet Archaeology:**
    - Paper intake forms, invoice receipts, and manual Excel tracking sheets.
    - *Rule of Thumb:* Every column in an active Excel sheet represents an unfulfilled requirement of the previous software system!
- **The "Archaeology Trap":**
  - Distinguish between **essential business logic** (must preserve) and **historical technical workarounds** (must deprecate, not blindly digitize).

# Choosing Elicitation Techniques: A Comparative Matrix

| Technique | Primary Strengths | Main Limitations | Best Applied When... |
|---|---|---|---|
| **Interviews** | High depth; builds personal stakeholder rapport | Vulnerable to bias; misses tacit knowledge | Initial discovery & understanding broad goals |
| **Workshops (JAD)** | Resolves conflicting priorities; rapid alignment | Scheduling friction; dominant personalities | Multiple stakeholders have competing requirements |
| **Ethnography** | Discovers actual behavior & informal workarounds | Highly time-consuming; non-generalizable | Complex, physical, high-stress work environments |
| **User Stories & Scenarios** | Testable; directly feeds agile sprint backlogs | Can miss global non-functional constraints | Defining incremental user-facing features |
| **Doc Analysis** | Highly objective; uncovers legacy business rules | Documents are often outdated or aspirational | Replacing legacy systems & regulated domains |

# THE 4 REQUIREMENTS ELICITATION TECHNIQUES

## 1. Interviews & Workshops




## 2. Ethnography & Observation




## 3. Scenarios & Storyboards




## 4. Document Archaeology



# Concept Check Question 4

During a requirements elicitation interview, an executive insists: "*Our clinical software must include a blockchain ledger to record patient vitals.*" What is the most effective engineering interview heuristic to apply?

- **A.** Apply the "5 Whys" to investigate the underlying data integrity and audit problem rather than prematurely locking in the suggested technology.
- **B.** Immediately begin drafting smart contracts and relational database schemas for the requested blockchain feature.
- **C.** Reject the executive's request outright because non-technical stakeholders are prohibited from proposing system capabilities.
- **D.** Politely terminate the interview and switch exclusively to passive workplace observation.



# Concept Check Question 5

In requirements engineering, why is **Ethnography (workplace observation)** uniquely vital when analyzing complex operational environments such as hospital emergency rooms or air traffic control?

- **A.** It uncovers tacit knowledge—ingrained habits, physical workarounds, and unwritten shortcuts that users perform automatically but never mention during interviews.
- **B.** It automatically compiles natural language requirements directly into executable acceptance test suites without human intervention.
- **C.** It eliminates the need for subsequent software architecture design, database modeling, or code review phases.
- **D.** It guarantees that the resulting software requirements will achieve 100% mathematical completeness on the initial development sprint.



# From Scenarios to Use Cases: Bridging Elicitation to Modeling

> *How do engineers transition from raw elicitation stories (Technique 3) into structured system models?*

- **Connecting to Elicitation Technique 3 (Scenarios & User Stories):**
  - Elicitation produces concrete human stories ("*Alice orders sushi using a coupon, but her card expires*").
- **What is a Scenario?**
  - A single, concrete execution path through a user interaction (one specific narrative sequence).
- **What is a Use Case?**
  - A **generalized** umbrella that aggregates **all related scenarios** (the Happy Path plus all alternative & exception flows) for a specific actor goal (*Place Food Order*).
- **Why Transition from Scenarios to Use Cases?**
  - Scenarios provide rich narrative context; use cases structure them into comprehensive functional contracts for engineering and test planning.

# Discovering Use Cases: The 5-Step Process

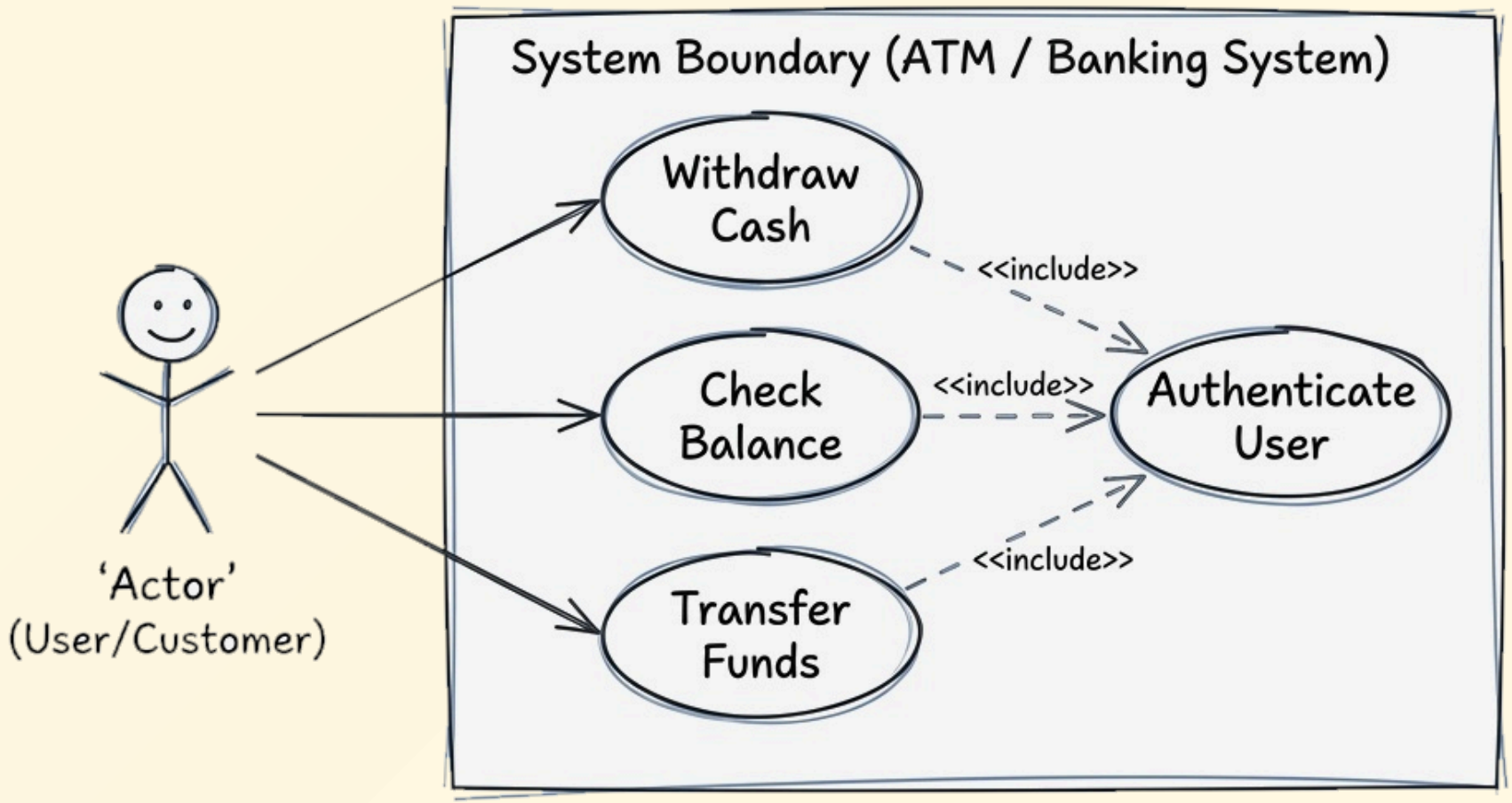
How do engineers systematically discover and structure use cases during elicitation?

1. **Identify External Actors:** Determine who or what interacts directly with the software (human roles, payment gateways, automated sensors).
2. **Identify Actor Business Goals:** For each actor, ask: "*What observable value do they need the system to deliver?*"
3. **Group Scenarios into Use Cases:** Cluster related user stories into verb-noun operational goals (*Place Order*, *Track Delivery*).
4. **Define System Boundaries:** Clearly demarcate what logic resides inside the application vs. what remains external or manual.
5. **Factor Common & Conditional Logic:** Extract shared mandatory steps (`<<include>>`) and optional variations (`<<extend>>`).

# Use Case Diagram Notation (UML)

- **Actors:** External entities (users or systems) interacting with the application.
- **Use Cases:** Ellipses representing high-level functional interactions.
- **System Boundary:** Box defining scope of the system under design.
- **Relationships:**
  - `<<include>>`**:** Behavior that is always executed as part of the base use case (e.g., *Authenticate User* included in *View Medical Records*).
  - `<<extend>>`**:** Optional or conditional behavior triggered under specific conditions (e.g., *Prescribe Controlled Substance* extends *Prescribe Medication*).

# Anatomy of a UML Use Case Diagram




System Boundary (ATM / Banking System)

# USE CASE DESCRIPTION: WITHDRAW CASH

## METADATA

- Primary Actor: Bank Customer
- Preconditions: ATM online
  - Cash vault loaded
  - Card inserted & validated
- Postconditions: Cash dispensed
  - Account debited
  - Audit log recorded
  - Card returned

## MAIN SUCCESS SCENARIO (HAPPY PATH)

1. Customer selects 'Withdraw Cash' and inputs amount ($100)
2. System verifies available account balance with Core Banking API
3. System confirms internal ATM vault has sufficient bills
4. System debits customer account & creates transaction log
5. System initiates cash dispensing mechanism
6. System ejects ATM card and sounds chime
7. Customer removes card; System opens cash dispenser shutter
8. Customer takes cash; System prints transaction receipt

## EXTENSIONS & EXCEPTIONS

2a. Insufficient Balance: System displays alert, offers balance check, and aborts
3a. Machine Out of Cash: System alerts customer, limits withdrawal, or aborts
6a. Unclaimed Card: If card not taken within 30s, ATM retracts card for security
8a. Unclaimed Cash: If cash left in tray for 30s, shutter closes and debit is reversed

# Concept Check Question 6

A software engineering team creates a UML Use Case Diagram showing an actor connected to the "Withdraw Cash" use case. Why must engineers author a detailed textual **Use Case Description** in addition to the diagram?

- **A.** Diagrams only show high-level scope; descriptions define sequential flows, preconditions, postconditions, and exception handling.
- **B.** UML diagrams cannot be rendered by web browsers without accompanying markdown text.
- **C.** Compilers require use case descriptions to allocate heap memory for actor threads.
- **D.** Descriptions convert non-functional requirements into automated GUI wireframes.



# Interactive Activity: Cybercab Requirements Elicitation (Pair Discussion)

> ***10-Minute Pair Mission:*** *You are Lead Requirements Engineers for Cybercab. Deliver a* ***3-Point Elicitation Action Plan*** *on paper / shared doc within 10 minutes:*

## Deliverable 1: Stakeholders (3m)

*Select 2 contrasting stakeholder groups:*

- **Action Guide:**
  - Pick 2 groups: e.g. Daily Rider, Car Owner (Lessor), Disabled Rider, or City Traffic Dept.
  - **Your Output:** Formulate **1 hidden / unstated pain point** for each group that a regular online survey would miss.

## Deliverable 2: Techniques (4m)

*Match 1 elicitation technique to each:*

- **Action Guide:**
  - Choose methods from 3.4: Contextual Observation, Cabin VR Mockup, or Workshops.
  - **Your Output:** Justify *why* this specific technique uncovers their tacit needs and emotional friction effectively.

## Deliverable 3: Reconcile (3m)

*Resolve 1 direct clash between them:*

- **Action Guide:**
  - Define 1 conflict: e.g. Curbside drop-off anywhere vs. City traffic safety laws.
  - **Your Output:** Apply **MoSCoW** (Must-Have vs. Won't-Have) to declare the winner and give your engineering rationale.

# 3.5 Requirements Validation & Change Management

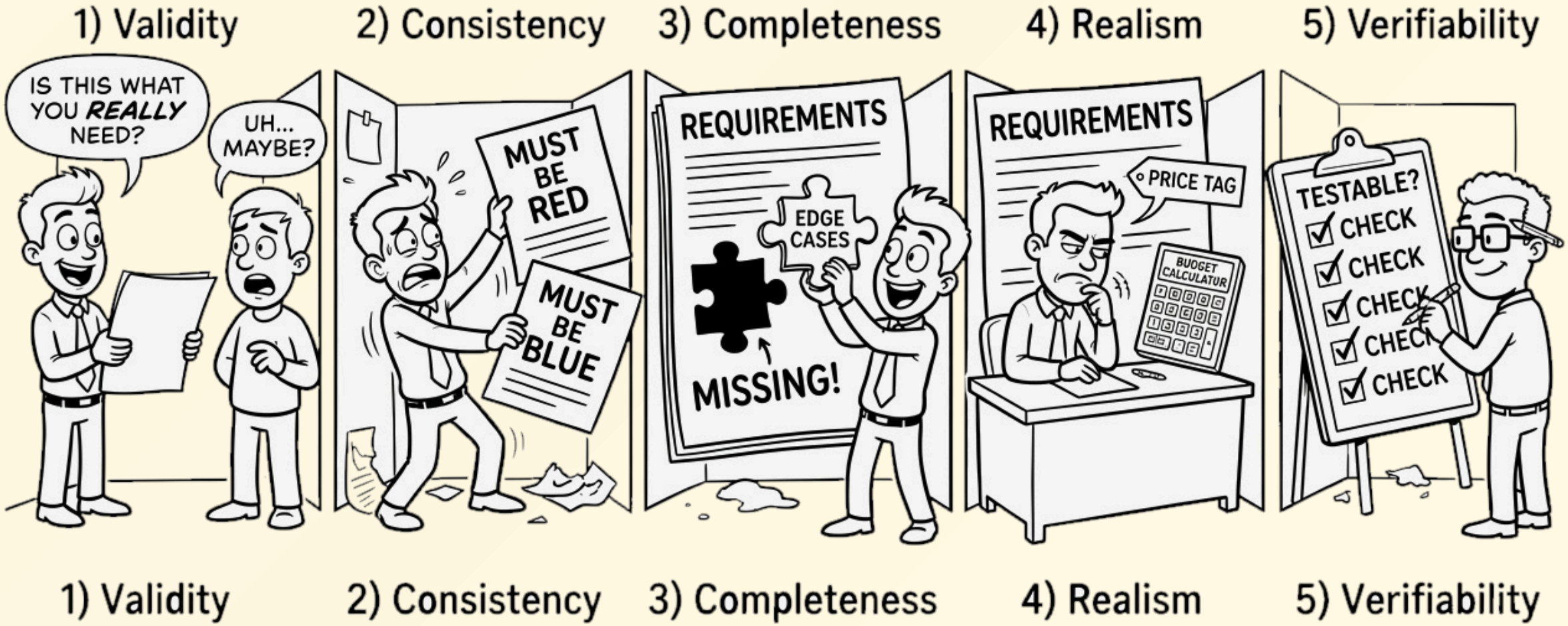
> *"Change is inevitable. In a living system, requirements management is the art of embracing change without descending into chaos."*

# Requirements Validation & Change Management

> *The critical quality process of certifying that requirements represent real user needs, and managing evolving specifications throughout the software lifecycle.*

- **High Cost of Requirements Errors:**
  - Fixing a requirements error after system delivery can cost **up to 100 times** more than fixing a coding bug!
- **5 Requirements Checks:**
  1. **Validity:** Does it reflect the real needs of the customer?
  2. **Consistency:** Are there conflicting requirements?
  3. **Completeness:** Are all requirements and constraints included?
  4. **Realism:** Can the requirement be implemented with current budget/tech?
  5. **Verifiability:** Can the requirement be objectively tested?

# The 5 Pillars of Requirements Validation



# Interactive Activity: 5-Check Requirements Audit (Pair Discussion)

> ***Scenario:*** *Audit this draft requirement: "The app shall deliver orders instantly and easily. Couriers must accept all incoming delivery tasks immediately, but may reject any order freely. Orders must arrive within 30 minutes under all conditions, even severe typhoons."*

## Round 1: Human Peer Audit (3 Mins)

*Apply the 5 Checks with your partner:*

- **Validity:** Does this satisfy genuine customer & business needs?
- **Consistency:** Identify direct logical contradictions in the rules.
- **Completeness:** Flag missing edge cases (spills, closures, refunds).
- **Realism:** Is 30-min typhoon delivery physically or legally viable?
- **Verifiability:** Spot subjective adjectives ("*instantly*", "*easily*").

## Round 2: AI Audit & Comparison (3 Mins)

*Run an AI audit and compare findings:*

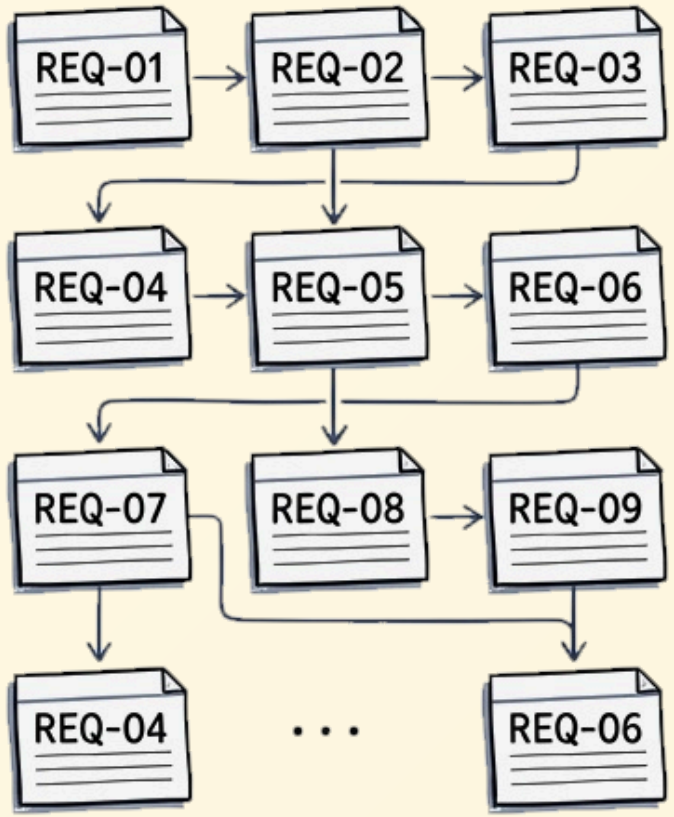
- **Prompt LLM:** "*Audit this requirement against the 5 SE checks (Validity, Consistency, Completeness, Realism, Verifiability): [Draft Text]*"
- **Compare Human vs. AI:**
  - **Agreement:** Did AI catch the same contradiction and vague terms?
  - **AI Blindspots:** Did AI miss safety regulations or human context?
  - **AI Strengths:** Did AI uncover edge cases your pair overlooked?

# Requirements Change & Management

- **Why Requirements Change:**
  - Business goals shift, new regulations emerge, payors and actual system users have conflicting demands.
- **Requirements Management Planning:**
  - **Unique Identification:** Tagging every requirement with a traceable ID (e.g., `REQ-SEC-012`).
  - **Traceability Policies:** Maintaining links from requirements to architecture modules, code commits, and test cases.
  - **Change Control Process:** Formal assessment of the cost and architectural impact before approving changes.

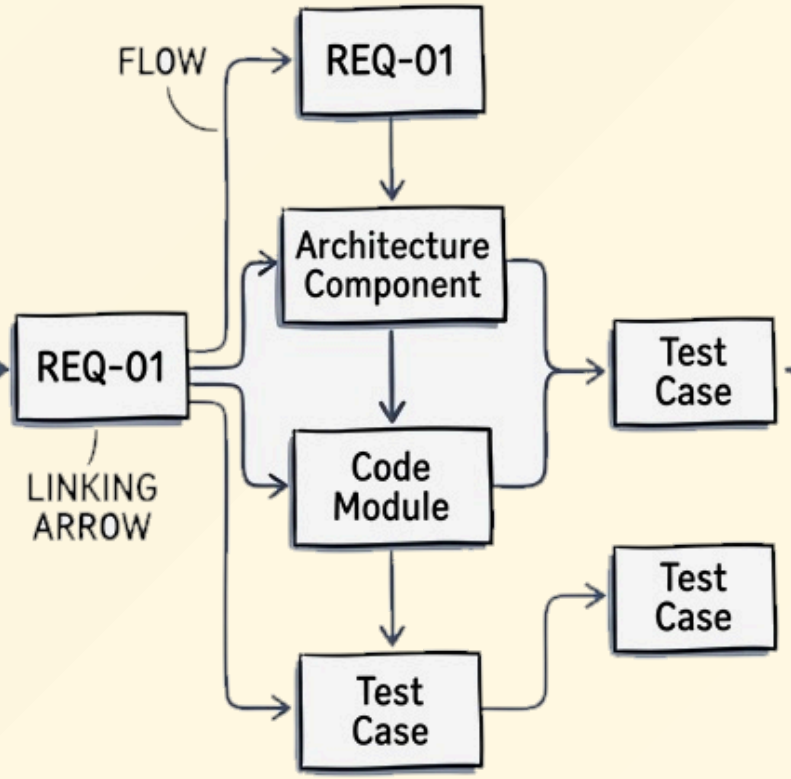
# REQUIREMENTS CHANGE MANAGEMENT & TRACEABILITY

## UNIQUE ID TAGGING



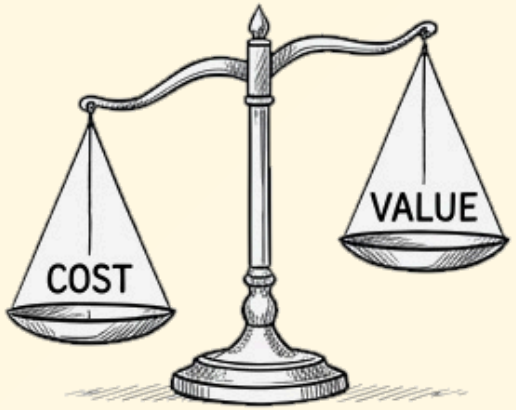

LINILY PRESENT ON
A DIRECT FLOW

## TRACEABILITY MATRIX




REQUIREMENTS SAFELY THROUGH CAUSING CHANGES

## CHANGE CONTROL BOARD & IMPACT ANALYSIS







**IMPACT ANALYSIS**

- Potential changes before approved
- Evaluatal changes before approved
- Evaluate changes before approved

# Concept Check Question 7

Why is fixing a requirements error after software delivery significantly more expensive than fixing an error during early development?

- **A.** Requirements documents cannot be legally modified once signed by clients.
- **B.** A late fix requires redesigning, recoding, retesting, and redeploying cascading components that were built on the flawed premise.
- **C.** Compilers automatically lock code repositories against changes after the first production release.
- **D.** Automated unit tests lose their validity after code has been deployed to cloud environments.



# 3.6 AI-Assisted Requirements Engineering

> *"AI is a cognitive amplifier for requirements analysis, but human judgment remains the irreplaceable anchor of accountability."*

# AI in Requirements Engineering: Overview

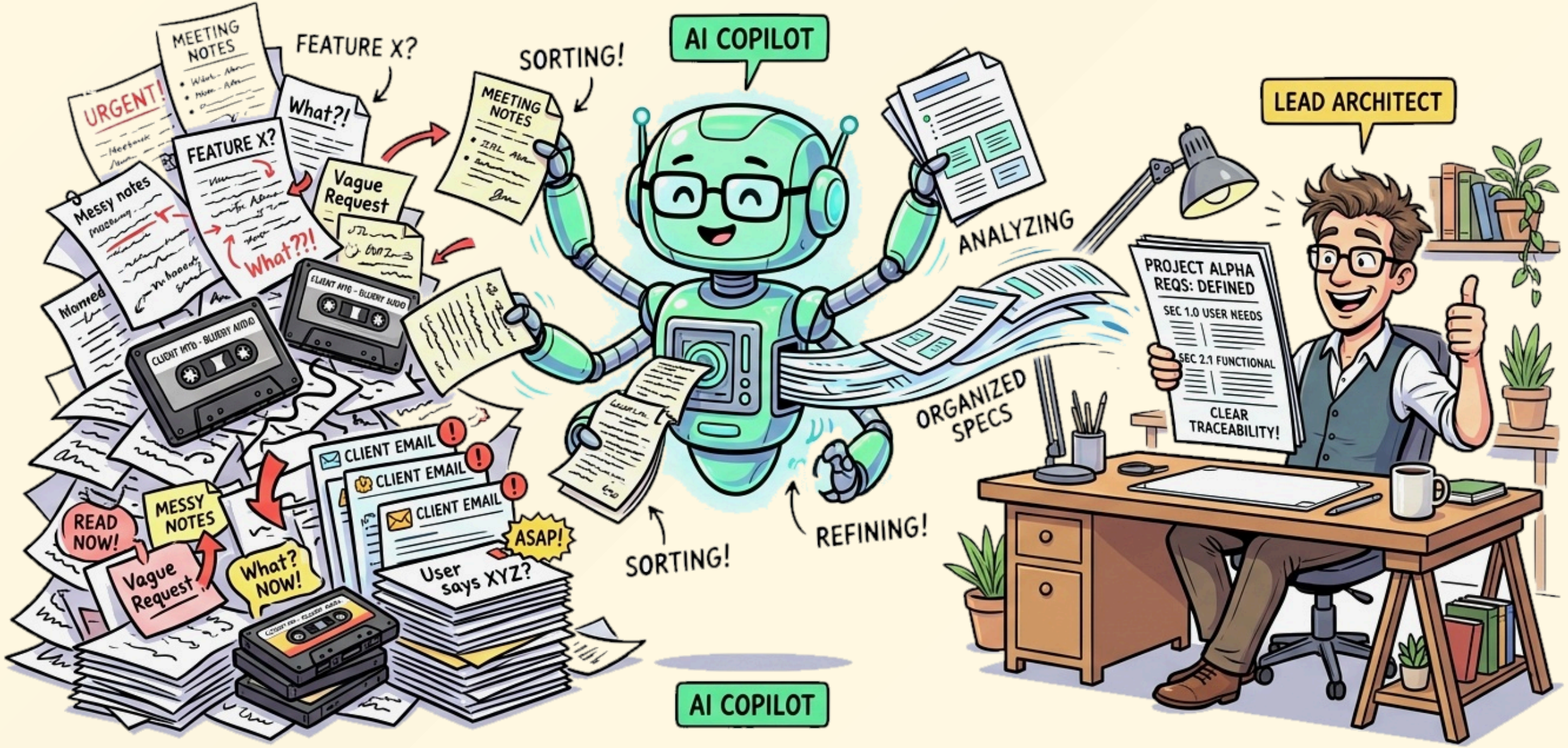
> *Leveraging Large Language Models as intelligent analytical copilots to bridge informal human dialogue and formal, testable software specifications.*

- **The Generative AI Paradigm Shift:**
  - LLMs and Generative AI tools serve as an **intelligent RE Copilot** across the software engineering lifecycle.
  - Bridges the gap between informal customer language and formal software engineering artifacts.
- **Key Capabilities:**
  - Drafting structured specifications from messy meeting notes.
  - Detecting semantic contradictions and missing edge cases across hundreds of pages of documentation.
  - Generating initial test cases and Gherkin acceptance criteria automatically.

# Why LLMs Work for RE: Pre-Trained Domain Ontologies

> *"When you ask an AI to design a food delivery app, it doesn't start from scratch—it already knows the business domain!"*
>
> ”

- **The Secret of Pre-Trained Domain Knowledge:**
  - LLMs are trained on billions of tokens: industry documentation, enterprise case studies, open-source codebases, and ISO/IEEE standards.
  - The model already possesses a latent **Domain Ontology**:
    - **In Banking:** It already knows accounts require transaction atomicity, multi-factor auth, and regulatory audit ledgers.
    - **In E-Commerce / Delivery:** It already understands multi-party split payments, geofencing, dynamic surge pricing, and cancellation refund rules.
- **The "Instant Domain Advisor" Advantage:**
  - **Surfacing Tacit Requirements:** Immediately proposes standard industry edge cases that junior engineers often forget (e.g., promo-code fraud, cold-chain temperature thresholds).
  - **Rapid Scaffolding:** Transforms informal stakeholder interview transcripts into Given-When-Then user stories and UML candidate entities in seconds.

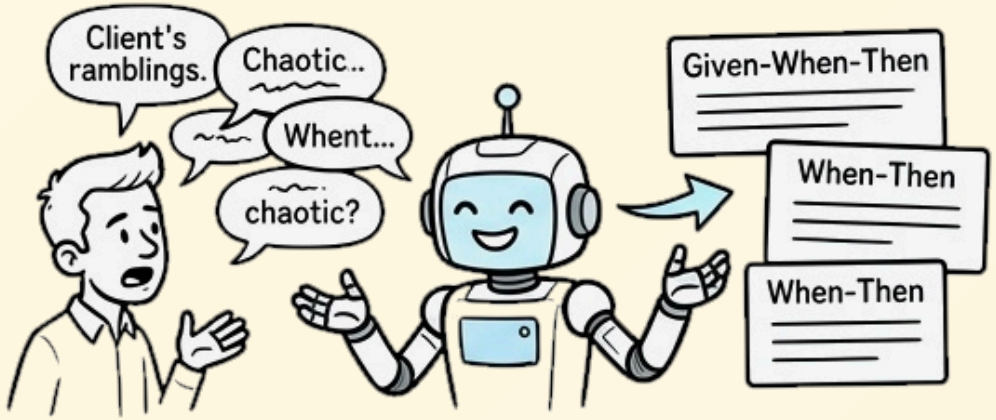

MEETING NOTES
URGENT!
FEATURE X?
What?!
FEATURE X?
What?!!
Vague Request
Messy notes
Planned
CLIENT MTG - BLUEBERRY MUFFIN
CLIENT MTG - BLUEBERRY MUFFIN
CLIENT EMAIL
CLIENT EMAIL
CLIENT EMAIL
ASAP!
User says XYZ?
READ NOW!
MESSY NOTES
Vague Request
What? NOW!
SORTING!
AI COPILOT
MEETING NOTES
ANALYZING
SORTING!
REFINING!
ORGANIZED SPECS
LEAD ARCHITECT
PROJECT ALPHA REQS: DEFINED
SEC 1.0 USER NEEDS
SEC 2.1 FUNCTIONAL
CLEAR TRACEABILITY!
AI COPILOT

# 4 Core AI Applications in RE

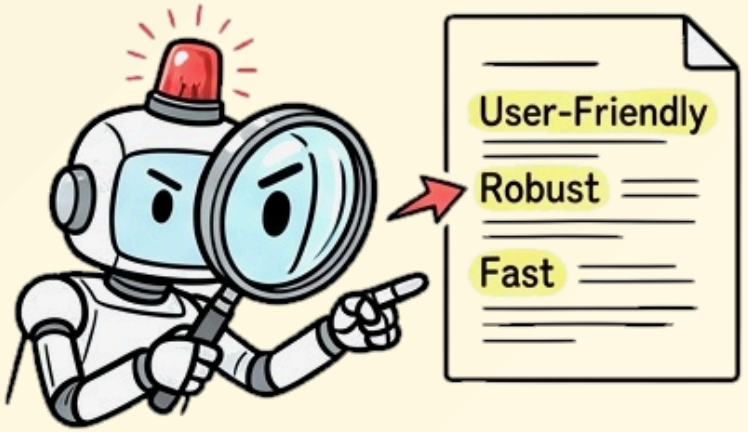
1. **Automated User Story & Specification Generation:**
   - Converts raw client interview transcripts into structured `Given-When-Then` Gherkin acceptance criteria.
2. **Ambiguity & Inconsistency Detection:**
   - Scans hundreds of requirement statements to detect contradictory terms and passive-voice vagueness.
3. **Traceability Matrix Construction:**
   - Uses semantic embeddings to map high-level user requirements to code functions and unit tests.
4. **Synthetic Stakeholder Persona Simulation:**
   - Simulates diverse user personas to brainstorm overlooked edge cases during early discovery.

# The 4 Core AI Applications in Requirements Engineering

## User Story Gen




## Ambiguity Detective



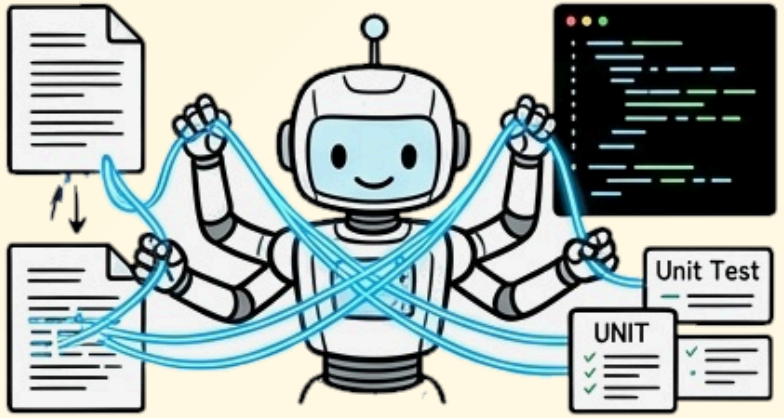

## User Story Gen

## Ambiguity Detective

## Persona Simulator



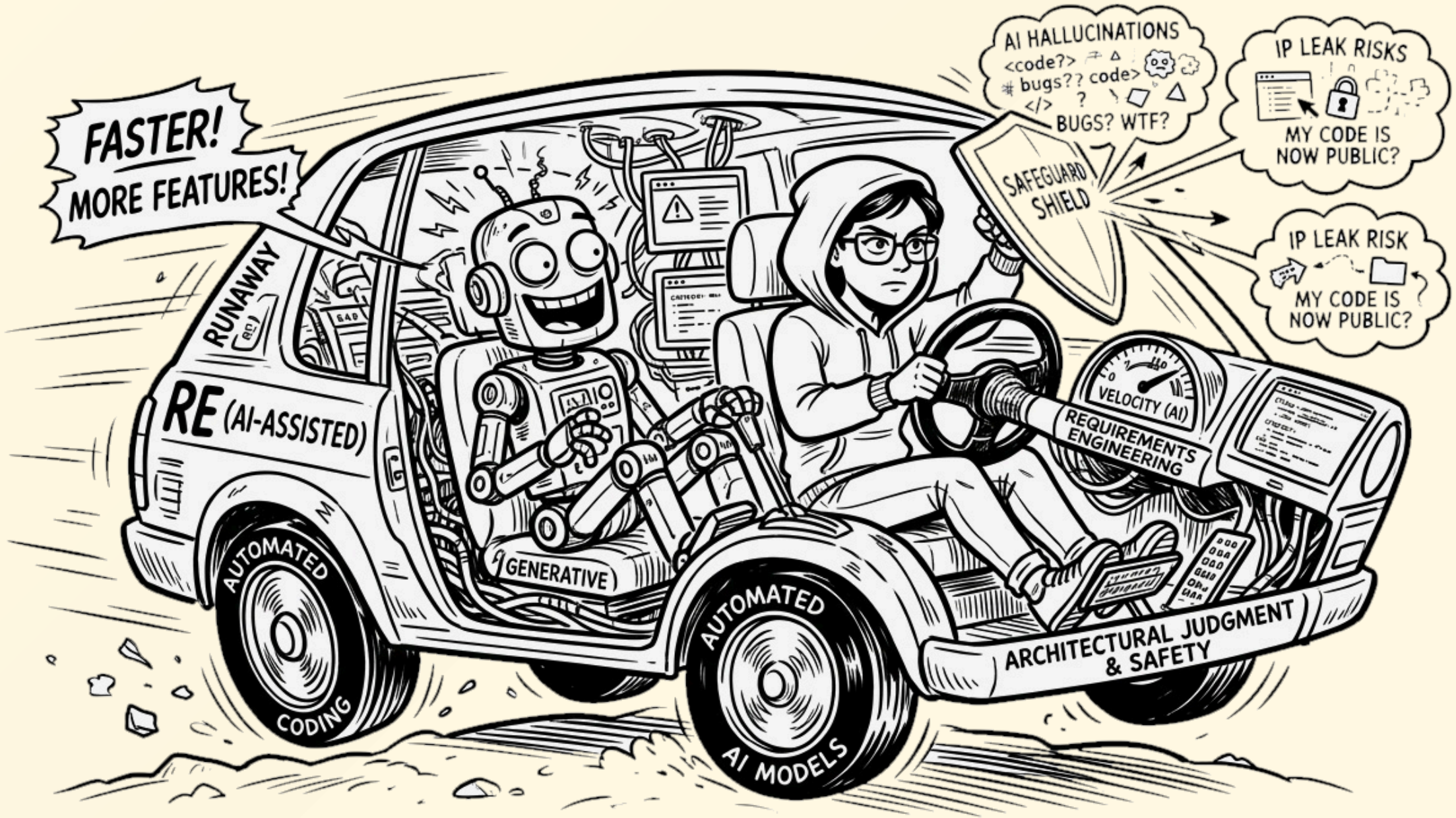
## Traceability Matrix



# Human-in-the-Loop: Risks & Best Practices

- **Risks of Unchecked AI in RE:**
  - **Hallucinations:** Inventing non-existent business rules or domain constraints.
  - **Privacy & IP Leaks:** Accidentally leaking proprietary business logic to public cloud models.
  - **Bias Amplification:** Reinforcing historical biases in requirements datasets.
- **The Golden Rule: Human-in-the-Loop:**
  - AI proposes; **the Human Software Engineer disposes.**
  - The human engineer retains legal, ethical, and architectural accountability for every accepted requirement.

# HUMAN-IN-THE-LOOP: THE ULTIMATE RESPONSIBILITY IN AI-ASSISTED RE



# Concept Check Question 8

When software engineering teams use Large Language Models (LLMs) to generate requirements specifications from stakeholder interview transcripts, what is the primary operational risk requiring human-in-the-loop verification?

- **A.** The LLM may hallucinate plausible-sounding but fictitious business logic, omitted edge-case constraints, and non-existent external API integrations.
- **B.** The LLM cannot output text formatted in Markdown bullet points or standard user story Given-When-Then acceptance criteria.
- **C.** The LLM will consume excessive server memory and cause database deadlock errors across production microservice clusters.
- **D.** The LLM strictly enforces waterfall development practices and refuses to generate requirements for iterative agile sprint cycles.



# 3.7 Conceptual Recap & References

> *"Requirements are the bridge between human intention and digital reality."* „

# Conceptual Recap: Fill-in-the-blank Quiz

Test your understanding of the core concepts in this chapter:

1. ___ requirements describe what the system should do, while ___ requirements specify system constraints.
2. High-level requirements written for customers are called ___ requirements.
3. Detailed specifications written as a technical contract for developers are ___ requirements.
4. The Goal-Question-Metric approach turns vague stakeholder wishes into ___ metrics.
5. In UML use cases, behavior that is always executed as part of a base use case uses the ___ relationship.
6. When using AI in requirements engineering, the engineering principle that ensures accountability is ___.

# References & Further Reading


- **Foundational Textbooks & Standards:**
  - Sommerville, I. (2016). *Software Engineering* (10th ed.). Chapter 4: Requirements Engineering. Pearson.
  - IEEE Std 830-1998 / ISO/IEC/IEEE 29148:2018: *Systems and software engineering — Life cycle processes — Requirements engineering*.
  - Cockburn, A. (2000). *Writing Effective Use Cases*. Addison-Wesley.
  - Wiegers, K., & Beatty, J. (2013). *Software Requirements* (3rd ed.). Microsoft Press.
- **AI & Empirical Requirements Engineering Research:**
  - Ferrari, A., et al. (2023). "Detecting Ambiguities in Requirements with Large Language Models." *IEEE RE'23*.
  - Arora, C., et al. (2024). "Automating Requirements Elicitation and Analysis with LLMs: An Empirical Evaluation." *ACM TOSEM*.
  - Zhang, H., Rahman, M., & Wang, Y. (2024). "Faithfulness, Hallucination, and Risk in Generative Requirements Engineering." *Empirical Software Engineering (EMSE)*.